CS 330 Final Project: Design Decisions

Nicole Snyder

Southern New Hampshire University

CS-330: Comp Graphics and Visualization

Dr. Vivian Lyon

October 13, 2026

CS 330 Final Project: Design Decisions

Development Choices

I designed the final scene as a compact desktop workspace, so the objects would be recognizable while remaining practical to build from low-polygon primitives. The scene includes a textured desk surface, articulated desk lamp, notebook, mug, and pencil cup with pencils. The lamp is the most complex object and combines cylinders, a sphere, and box meshes for the base, vertical support, hinge, and angled head. The mug combines a cylinder and torus, while the pencil cup uses a cylinder, and the pencils use cylinders with cone tips. I considered using more complex custom meshes, but reusable primitives kept the polygon count manageable while still producing recognizable objects and made individual parts easier to reposition or resize. Texture and material choices further separate the surfaces. The desk uses a rustic wood texture, the lamp uses gold and brushed-gold textures, and the mug and pencil cup use a stainless texture. I adjusted UV scaling to preserve the proportions of the texture images and reduce visible repetition or stretching. For lighting, I used a warm spotlight near the lamp head, a cool colored point light for separation on the right side, and a softer neutral point light, so the left-side objects remain visible. Ambient, diffuse, and specular Phong components preserve the lamp as the visual focal point while keeping the rest of the workspace readable.

Navigation

The camera controls allow a user to inspect the scene from multiple positions without losing a clear sense of orientation. In ProcessKeyboardEvents(), W and S move forward and backward, A and D move left and right, and Q and E provide vertical movement. These inputs call the camera's ProcessKeyboard() function with directional values and gDeltaTime, keeping

movement smooth and independent of frame rate. Mouse movement controls yaw and pitch so the user can look up, down, left, and right without changing the camera position. The mouse wheel adjusts travel speed, giving the user finer control near an object and faster movement when crossing the scene. The M key toggles between perspective and orthographic projection, while P selects perspective and O selects orthographic directly. PrepareSceneView() uses the same camera view matrix while switching the projection calculation between glm::perspective() and glm::ortho(), so changing projection does not reset the user's position or orientation. This makes it possible to compare the same arrangement in a depth-based 3D view and a parallel 2D-style view.

Custom Functions and Modularity

The project is organized around functions that separate scene setup, object rendering, lighting, and camera input. SetupSceneLights() keeps lighting in one location, while ProcessKeyboardEvents() groups keyboard navigation and projection controls. PrepareScene() handles one-time texture, material, mesh, and lighting setup, while PrepareSceneView() updates timing, input, and view/projection matrices each frame. Object-specific functions such as RenderDeskSurface(), RenderDeskLamp(), RenderNotebook(), RenderMug(), and RenderPencilCup() isolate the transformations, textures, and materials for each object. RenderScene() coordinates those functions in sequence. Reusable helpers such as SetTransformations(), SetShaderTexture(), SetTextureUVScale(), and SetShaderMaterial() reduce repeated OpenGL setup code and make rendering functions easier to read. These helpers and object-rendering functions could be reused in another OpenGL scene because their responsibilities are separated from the main rendering loop. Placing every draw call directly in

RenderScene() would have increased duplication and made revisions harder. The modular structure instead lets one object, control, or light be revised without disturbing unrelated code.